Journal of Complex Networks (2015) **3**, 249–263
doi:10.1093/comnet/cnu033
Advance Access publication on 1 August 2014

Modelling the neighbour aid phenomenon for installing costly complex networks

Alexander Gutfraind[†]
University of Illinois at Chicago, 1603 W. Taylor St, Chicago, IL 60612, USA
[†]Corresponding author. Email: agutfraind.research@gmail.com

Milan Bradonjić
Mathematics of Networks and Systems, Bell Labs, Alcatel-Lucent, 600 Mountain Avenue, Murray Hill, NJ 07974, USA

and

Tim Novikoff
Center of Applied Mathematics, Cornell University, Ithaca, NY 14853, USA

Edited by: Aaron Clauset

[Received on 16 January 2014; accepted on 27 June 2014]

Complex networks are frequently used to model natural, social and engineered systems. A lot of studies looked at the problem of modelling the topology of a network or designing it. Less is known, even in a stylized form, about how one might optimally install a given network topology, assuming that installing a node aids the installation of neighbouring nodes. This problem arises in education where it is desired to teach mutually reinforcing words, as well as in engineering whenever damaged infrastructure networks are to be recovered. We introduce a new discrete optimization problem where the goal is to minimize the total cost of installing a given network. This cost is determined by the structure of the network and the sequence with which the nodes are installed. Namely, the cost of installing a node is a function of the number of its neighbours that have been installed before it. We analyse the common case where the cost function is decreasing and convex, and provide bounds on the cost of the optimal solution. We also show that all sequences have the same cost when the cost function is linear and give results on the cost of a random solution for several models of random graphs. Examining the computational complexity, we show that the problem is NP-hard when the cost function is arbitrary. Finally, we provide an integer programme, a dynamic programming algorithm, and greedy heuristics which give high quality solutions.

Keywords: cognitive network; infrastructure network; disaster recovery; permutation optimization; linear ordering problem; neighbour-aided network installation problem.

1. Introduction

Consider the problem of designing educational software that teaches students vocabulary words. As students becomes familiar with a word, it becomes easier to learn additional words that are related to it. For example, if the students understand the word *epigraph*, they would more easily learn the etymologically adjacent words *monograph* and *epitope*. This means that the sequence with which words are introduced

© The authors 2014. Published by Oxford University Press. All rights reserved.

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

could be optimized to make them easier to learn. Overall, the vocabulary could be organized into a complex network of nodes (words) linked through their roots. Abstractly, as the students advance, previously learned nodes assist with learning adjacent nodes—a phenomenon that we call 'neighbour aid'.

For an entirely different application of neighbour aid, consider infrastructure systems which could be represented as complex networks, including transport, communication and power networks. Large parts of these networks might become inoperable following natural disasters to the extent that the cost of the recovery might be measured in hundreds of billions of dollars. When faced with vast devastation, authorities must develop a plan or schedule for restoring the networks. A particularly challenging aspect of the recovery is the lack of surviving infrastructure, such as roads or power, necessary to support the recovery operations. For example, to clear and rebuild roads, equipment must be brought in, but many of the access junctions are themselves blocked and damaged. What is the optimal schedule? This optimization challenge has the same abstract flavour as the educational software problem, although the practical problems have many unique aspects.

Here, we introduce a description of the neighbour aid phenomenon as a discrete optimization problem, which we call the *Neighbour-Aided Network Installation Problem* (NANIP). For simplicity of analysis, we propose that during the learning/reconstruction of a network all of its nodes and edges are to be learned/re-installed. We assume that the cost of the edges is either irrelevant (as in the case of vocabulary) or not determined by the schedule and examine the following question: How could the recovery schedule be optimized in order to reduce the total cost?

NANIP offers a new stylized model for disaster recovery of networks but the interest in disaster recovery of networks is not new. A partial list of existing studies include [1] which considers a damaged network carrying flows, and the objective is to prioritize a budget of work crews to recover damaged relay nodes (see also a related model in [2]). The analyses in [3] similarly model infrastructure systems as a set of interdependent network flows, and formulate the problem of minimizing the cost of repairing such damaged networks (see also [4,5]). Another class of models is exemplified by [6] who introduce the Power System Stochastic Storage Problem (PSSSP). PSSSP considers how to stockpile components for restoring a power distribution system resources in preparation for likely damage from hurricanes. The authors in [7] develop a stochastic optimization problem for stockpiling resources and then distributing them following a disaster. Additionally, they review a number of existing studies in this field. Educational software is a much newer area, with only a few existing models [8]. A network recovery such as in NANIP is conceptually similar to influence maximization problems [9]: in both cases the aim is to switch the nodes into a state we call installed with the least cost, and the submodular influence function is related to the convex case in NANIP.

More abstract problems related to NANIP are the single processor scheduling problem [10], the linear ordering problem [11] and the study of tournaments in graph theory [12]. Also related is the travelling salesman problem (TSP) [13]. NANIP assumes that certain tasks are dependent and cannot be performed in parallel, but unlike many scheduling problems, there are no partial order constraints. NANIP asks for a schedule based on a graph structure, but unlike with TSP, the cost associated with visiting a given node could depend on *all* of the nodes visited before the given node.

2. General formulation and basic results

An instance of NANIP is specified by a network $G = (V, E)$ and a real-valued function $f : \mathbb{N}_0 \to \mathbb{R}_+$. The domain of f are non-negative integers up to the degree of the highest degree node in G. The goal is to find a permutation of the nodes that minimizes the total cost of the network installation. The cost of

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

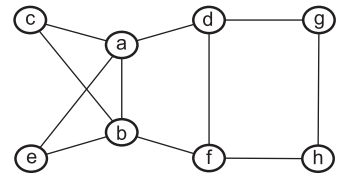


FIG. 1. An illustration of the problem on a simple instance. When $f(k) = 12/(1+k)$, nodes a and b are best installed early because they can reduce the cost of many of their neighbours. An optimal solution is the sequence $\sigma = (a, b, c, e, d, f, g, h)$ with cost $46 = 12 + 6 + 4 + 4 + 6 + 4 + 6 + 4$.

installing node $v_t \in V$ under a permutation σ of V is given by

$$f(r(v_t, G, \sigma)),$$

where $r(v_t, G, \sigma)$ is the number of nodes adjacent to v_t in G that appear before v_t in the permutation σ. The total cost of installing G according to the permutation σ is given by

$$C_G(\sigma) = \sum_{t=1}^{n} f(r(v_t, G, \sigma)). \tag{1}$$

The problem is illustrated in Fig. 1.

NANIP could also be expressed as optimization over n-by-n permutation matrices Π where A is the adjacency matrix of G

$$\min_{\Pi} \sum_{t=1}^{n} f\left(\sum_{i=1}^{t-1} W_{t,i}\right), \tag{2}$$

where we define $W := \Pi^T A \Pi$. The inner sum computes the number of nodes adjacent to node v_t that are installed before time t. While this matrix formulation might be of interest for certain investigations (e.g. numerical algorithms), we will use the original formulation in the rest of the paper. Throughout the paper, we use the standard notation $n := |V|$ and $m := |E|$.

We assume that G is connected and undirected, unless we note otherwise. If G has multiple connected components, NANIP could be solved on each component independently of the others without affecting the total cost. Generally, the choice of f depends on the application: f would rapidly reach a plateau if there is a physical limit to how many neighbours could be involved in the installation process. The opposite case of a gradually decreasing f might occur if the installation of node v is a complex process that involves all of the neighbours of v. To be precise, here and elsewhere in the paper, by decreasing we mean non-increasing, i.e. $f(i) \geqslant f(j)$ if $i > j$, and similarly, by increasing we mean non-decreasing.

We begin with a preliminary lemma which establishes that all the arguments used in calculating the node costs must sum to m, the number of edges in the network.

LEMMA 1 For any network G, and any permutation σ of the nodes of G,

$$\sum_{t=1}^{n} r(v_t, G, \sigma) = m. \tag{3}$$

Proof. For every edge in G and any permutation σ, one of the two endpoints of the edge is the latter of the two to be installed under σ. By definition, $r(v_t, G, \sigma)$ is the number of edges whose latter endpoint

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

is installed at time t. Thus, summing up $r(v_t, G, \sigma)$ for all $t = 1, 2, \ldots, n$ accounts for all the edges in G. □

One application of this lemma is the case of a linear cost function $f(k) = ak + b$, for some real numbers a and b. With such a function the optimization problem is trivial in that all installation permutations have the same cost.

COROLLARY 2 For any G, when $f(k) = ak + b$ for some real numbers a and b, then $C_G(\sigma) = am + bn$.

Proof.

$$C_G(\sigma) = \sum_{t=1}^{n} f(r(v_t, G, \sigma)) = \sum_{t=1}^{n} (ar(v_t, G, \sigma) + b) = a \sum_{t=1}^{n} r(v_t, G, \sigma) + bn = am + bn.$$

□

3. Expected cost on random graphs

For some applications and for theoretical interest, we examine the cost of a randomly chosen solution on several models of random graphs. We begin by examining the Erdős–Rényi model [14], where each of all $\binom{n}{2}$ node pairs is independently connected with probability p.

LEMMA 3 Let $G_{n,p}$ be an Erdős–Rényi random graph on n nodes with the edge probability p. Let σ be a permutation of the nodes of $G_{n,p}$ chosen uniformly at random on the set of nodes. For any real-valued function $f : \mathbb{N}_0 \to \mathbb{R}_+$ and $p > 0$ the expected cost satisfies

$$\mathbb{E}(C_{G_{n,p}}(\sigma)) \leqslant \frac{1}{p} \sum_{k=0}^{n-1} f(k).$$

Notice that the graph does not need to be connected in Lemma 3.

Proof. Let $\sigma = (v_1, \ldots, v_n)$ be a permutation on n nodes chosen uniformly at random from the set of all such permutations, where the label v_i is assigned to the ith node to appear in σ. Then when v_t is installed at time t, the probability that exactly k adjacent nodes are already installed is $\binom{t-1}{k} p^k (1-p)^{t-1-k}$, which is the probability that exactly k nodes from the set $\{v_1, v_2, \ldots, v_{t-1}\}$ are adjacent to v_t in G. Hence the expected cost of installing v_t is

$$\sum_{k=0}^{t-1} \binom{t-1}{k} p^k (1-p)^{t-1-k} f(k),$$

for $t = 1, \ldots, n$. Thus, the expected total cost of installing G according to σ is

$$\mathbb{E}(C_{G_{n,p}}(\sigma)) = \sum_{t=1}^{n} \sum_{k=0}^{t-1} \binom{t-1}{k} p^k (1-p)^{t-1-k} f(k) = \sum_{k=0}^{n-1} \frac{p^k}{(1-p)^k} f(k) \sum_{t=k}^{n-1} \binom{t}{k} (1-p)^t.$$

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

For any $p \in (0,1)$ and any positive integer k, we have

$$\sum_{t=k}^{\infty} \binom{t}{k} (1-p)^t = \frac{(1-p)^k}{p^{k+1}}.$$

Truncating at $t = n-1$ and substituting this expression into $\mathbb{E}(C_{G_{n,p}}(\sigma))$ gives the desired inequality. □

An exact expression for $\mathbb{E}(C_{G_{n,p}}(\sigma))$ could be given by using the hypergeometric function ${}_2F_1(a,b,c,z) = \sum_{i=0}^{\infty}((a)_i(b)_i/(c)_i)(z^i/i!)$, where the failing factorial $(x)_i = x(x-1)\cdots(x-i+1)$ for $i \geqslant 0$. The expected total cost of installing $G_{n,p}$ according to σ can be computationally evaluated by using Mathematica 6.0 (Wolfram Research, Champaign, IL, USA), 2007

$$\mathbb{E}(C_{G_{n,p}}(\sigma)) = \frac{1}{p}\sum_{k=0}^{n-1} f(k) - \sum_{k=0}^{n-1} f(k) \binom{n}{k} p^k (1-p)^{n-k} {}_2F_1(1, n+1, n+1-k, 1-p).$$

We now consider a random graph with a given degree sequence $Q = (Q_0, Q_1, \ldots)$, where Q_d is the expected number of nodes that have degree $d \geqslant 0$. This model is also known as the configuration model and for the definition see the work in [15]. Because the degree sequence might be non-graphical or produce a multigraph, the lemma is formulated in such a way that only simple graphs are included in the expected cost.

LEMMA 4 Let $G_{n,Q}$ be a random graph with given degree sequence Q. Let σ be a permutation of the nodes of $G_{n,Q}$ chosen uniformly at random on the set of nodes. For any real-valued function $f : \mathbb{N}_0 \to \mathbb{R}_+$ the expected cost satisfies

$$\mathbb{E}(C_{G_{n,Q}}(\sigma)) = \sum_{t=0}^{n-1}\sum_{k=0}^{t}\sum_{d=0}^{n-1} Q_d \frac{\binom{t}{k}\binom{n-1-t}{d-k}}{\binom{n-1}{d}} f(k).$$

Proof. By linearity of expectation, the expected cost $\mathbb{E}(C_{G_{n,Q}}(\sigma))$ is equal to the sum over $t = 0, \ldots, n-1$ of the expected cost to install a node v_t at time t. The probability that v_t has degree d is Q_d. Given that degree of v_t is d, the number of already installed nodes, adjacent to v_t, follows the hypergeometric distribution $\binom{t}{k}\binom{n-1-t}{d-k} / \binom{n-1}{d}$. Averaging over all possible k and d, and summing up over t concludes the proof. □

The expression for the expected cost of a graph with given degree distributions simplifies in the case of a random d-regular random graph, for the definition see [16]. In this case, the graph sequence is equal to zero everywhere except at d and $Q_d = n$. Then we have the following remark.

REMARK 5 The expected cost to install a random d-regular graph G_n^d is given by

$$\mathbb{E}(C_{G_n^d}(\sigma)) = \binom{n-1}{d}^{-1} \sum_{t=1}^{n}\sum_{k=0}^{t-1} \binom{t-1}{k}\binom{n-t}{d-k} f(k).$$

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

This expected cost $\mathbb{E}(C_{G_n^d}(\sigma))$ can be evaluated further by using Mathematica 6.0 (Wolfram Research, Champaign, IL, USA), 2007.

$$\mathbb{E}(C_{G_n^d}(\sigma)) = \binom{n-1}{d}^{-1} \sum_{k=0}^{n-1} f(k) \left(\binom{-1}{d-k} \binom{n}{k} {}_2F_1(1+d-k, 1+n, 1-k+n, 1) + \binom{-1-k+n}{d-k} {}_2F_1(1+k, 1+d-n, 1+k-n, 1) \right).$$

4. Decreasing convex cost function

The case of a decreasing convex cost function f is natural for many applications of NANIP because many times the neighbour aid phenomenon declines in significance as the number of neighbours increases. For a decreasing cost function $f : \mathbb{N}_0 \to \mathbb{R}_+$, we say f is *decreasing convex* if $f(i) - f(i+1) \geqslant f(j) - f(j+1)$ for all $j \geqslant i$ in $\mathbb{N}_0$. The name 'decreasing convex' indicates that f is non-increasing and its *discrete first derivative* $f(i+1) - f(i)$ is non-negative in $i \geqslant 0$. This is the discrete analogue of decreasing convex twice-differentiable function, which is defined as a non-increasing function with a non-negative second derivative.

It is illuminating to see the effect of convexity in f for a small graph, such as a three-node path graph $V = \{v_1, v_2, v_3\}$ and $E = \{(v_1, v_2), (v_2, v_3)\}$. A sequence like (v_1, v_3, v_2) has cost $f(0) + f(0) + f(2)$ while a sequence like (v_1, v_2, v_3) costs $f(0) + f(1) + f(1)$. When f is decreasing convex, the second sequence is at least as expensive as the first one, since $f(1) + f(1) \leqslant f(0) + f(2)$.

For decreasing convex cost functions it is possible to find absolute lower bounds on the cost of installing a graph. To state this bound we extend the domain of f to non-integer values so that it is defined on all reals in $[0, n-1]$. One approach that preserves convexity is to define $f(q)$ for every positive non-integer q as the linear interpolation of $f(\lfloor q \rfloor)$ and $f(\lceil q \rceil)$. To derive the bound, we start with the case where only a subgraph H of the network needs to be installed.

THEOREM 6 Let V_H be a non-empty set of nodes $V_H \subset V_G$ and let $H = (V_H, E_H)$ be the induced subgraph of $G = (V_G, E_G)$. Let σ be any installation sequence for G such that it first installs $V_G \setminus V_H$ and then V_H. Denote with E_{GH} the cut from $V_G \setminus V_H$ to V_H :

$$E_{GH} = \{\{i, j\} \in E_G : i \in V_G \setminus V_H, j \in V_H\}. \tag{4}$$

Then for any decreasing convex cost function f, the total cost of installing V_H satisfies

$$C_H(\sigma) \geqslant |V_H| f\left(\frac{|E_H| + |E_{GH}|}{|V_H|} \right). \tag{5}$$

The partial installation scenario in Theorem 6 might be relevant to some network recovery situations. The graph G may represent the total network of a region while the subgraph H represents just the damaged subgraph. As a result, the cost of rebuilding H is reduced by connections to existing infrastructure (the edges E_{GH}).

Proof. The argument follows from Jensen's inequality

$$f\left(\frac{\sum_{i=A}^{B} x_i}{B - A + 1} \right) \leqslant \frac{\sum_{i=A}^{B} f(x_i)}{B - A + 1}, \tag{6}$$

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

Let $x_i = r(v_i, G, \sigma)$, $A = n + 1 - |V_H|$ and $B = n$. From the edge decomposition (Lemma 1), it follows that $C_H(\sigma) = \sum_{i=A}^{B} x_i = |E_H| + |E_{GH}|$. Rearranging Jensen's inequality now leads to the desired result. □

This bound suggests that the best installation sequence is one where the cost for every node is close to the average cost. A special case is where $V_G \setminus V_H$ is a single node. Its installation cost must be $f(0)$ and so we have a lower bound on the cost of G.

COROLLARY 7 Let $G = (V, E)$ be any graph and σ any installation sequence. If f is decreasing convex, then

$$C_G(\sigma) \geqslant f(0) + (n-1) f\left(\frac{m}{n-1}\right).$$

An immediate application of this bound is to find the optimal solutions for tree graphs. On a tree T, the bound gives $C_T(\sigma) \geqslant f(0) + (n-1)f(1)$ for any sequence σ. Consider any algorithm that picks a random starting node and then proceeds to install neighbours of installed nodes until all the nodes are installed. (An explicit example is stated later in the paper as Algorithm 2.) Such an algorithm would install T at a cost $f(0) + (n-1)f(1)$, which must be optimal by Corollary 7.

To derive more complex bounds, it is useful to draw on the theory of majorizations [17].

DEFINITION 8 For $x, y \in A \subset \mathbb{R}^K$, we say that x is majorized by y (denoted $x \prec y$) if, when both vectors are ordered in increasing order, $\sum_{i=1}^{k} x_i \geqslant \sum_{i=1}^{k} y_i$ for all $k = 1, \ldots, K-1$ and $\sum_{i=1}^{K} x_i = \sum_{i=1}^{K} y_i$.

DEFINITION 9 A real-valued function ϕ defined on $A \subset \mathbb{R}^K$ is called Schur-convex on A if for all $x \prec y$, $\phi(x) \leqslant \phi(y)$.

The cost of a installing a graph is a Schur-convex function when f is convex. To see that, recall that a sequence σ uniquely defines the in-degrees of all the nodes, $r(v, G, \sigma)$. Therefore, the cost function of a graph C_G could be viewed as a function on the in-degrees defined by σ.

LEMMA 10 For an instance of NANIP on G, if f is convex, then the graph cost function $C_G(\sigma)$ is Schur-convex.

This is an immediate application of that fact that f is convex and C_G is the sum $\sum_{v \in V_G} f(r(v, G, \sigma))$ (see [17, Proposition C.1, p. 64]).

We are now ready to give another bound on the cost of a graph by considering the degrees of the nodes. Assume that the nodes are labelled in order of increasing degree in G. Thus, if d_i gives the degree of node v_i, then $d_1 \leqslant d_2 \leqslant \cdots \leqslant d_n$. We consider the following relaxation of NANIP (1), in which we do not seek an installation sequence from which the values of $r(v_t, G, \sigma)$ are deduced, but rather we seek a collection of numbers, $x_1, \ldots, x_n$, which satisfy constraints that are satisfied also by $r(v_t, G, \sigma)$ in NANIP. More precisely for every i we require that x_i is less than or equal to d_i, and we require that values of $\sum_{i=1}^{n} x_i = m$ (compare with Lemma 1). Thus, we have the following minimization problem.

$$P = \min \quad \sum_{i=1}^{n} f(x_i), \tag{7}$$

$$\text{s.t.} \quad \sum_{i=1}^{n} x_i = m \tag{8}$$

$$0 \leqslant x_i \leqslant d_i \quad \text{and} \quad x_i \in \mathbb{N}_0 \ \text{ for } i = 1, 2, \ldots, n.$$

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

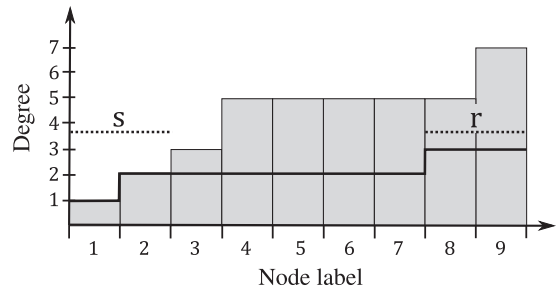


FIG. 2. An illustration of the lower bound of (10). Gray areas indicate the degrees of the nodes (d_i) while the solid line shows an optimal solution (x_i^*).

It is easy to find P exactly, giving an analytic lower bound on NANIP with decreasing convex f. Namely, P occurs when the first s of the nodes are installed with $x_i^* = d_i$ and the rest of the nodes are installed with x_i^* equal to d_s or $d_s + 1$, as follows.

THEOREM 11 For any G and decreasing convex f and any σ we have the following bound. Let $d_1 \leqslant d_2 \leqslant \cdots \leqslant d_n$ be the degrees of the nodes when the nodes are labelled in order of increasing degree. If $d_1 > m/n$, let $s = 0$, otherwise let

$$s = \max\left\{ k \in \{1, 2, \ldots, n\} : (n-k)d_k + \sum_{i=1}^{k} d_i \leqslant m \right\}. \tag{9}$$

When $s = 0$, any optimal solution x^* of (7) has $n - (m - n\lfloor m/n \rfloor)$ elements in $(x_i^*)_{i=1}^n$ equal to $\lfloor m/n \rfloor$, and $m - n\lfloor m/n \rfloor$ elements equal to $\lfloor m/n \rfloor + 1$; When $s \geqslant 1$, let $r = m - (n-s)d_s - \sum_{i=1}^{s} d_i$, and an optimal solution is given by the following assignment: (i) $x_i^* = d_i$ for $i = 1, 2, \ldots, s$; (ii) for $s < i \leqslant n$: $x_i^* = d_s + 1$ for exactly r indices and $x_i^* = d_s$ for exactly $n - s - r$ indices (illustrated in Fig. 2). Thus the cost of NANIP can be bounded from below by

$$C_G(\sigma) \geqslant P = \begin{cases} \left(n - \left(m - n\left\lfloor \frac{m}{n} \right\rfloor\right)\right) f\left(\left\lfloor \frac{m}{n} \right\rfloor\right) + \left(m - n\left\lfloor \frac{m}{n} \right\rfloor\right) f\left(\left\lfloor \frac{m}{n} \right\rfloor + 1\right), & \text{when } s = 0, \\ (n - r - s) f(d_s) + r f(d_s + 1) + \sum_{i=1}^{s} f(d_i), & \text{when } s \geqslant 1. \end{cases} \tag{10}$$

Proof. Consider x^* and any feasible solution $x \neq x^*$. We will show that $C_G(x^*) \leqslant C_G(x)$. It would be sufficient to show that for any feasible x, x majorizes x^* denoted by $x^* \prec x$, that is x^* lies in the convex hull of x and their elements have the same sum.

Let i be the smallest index where $x_i^* \neq x_i$. Case $x_i^* > x_i$: by Equation (8), $\sum_i x_i^* = \sum_i x_i$ and there must exist index $j > i$ where $x_j^* < x_j$. Since x_i^* is an increasing sequence ($x_i^* \leqslant x_j^*$) we obtain: $x_i < x_i^* \leqslant x_j^* < x_j$, i.e. $x_i < x_j$. Define a new sequence x' where $x' = x$ except at indices i and j, where $x_i' = x_i + 1$ and $x_j' = x_j - 1$. The operation $x' \to x$ produces a feasible solution because x and x^* are feasible and they bound the elements of x'. In the case $x_i^* < x_i$, we find $j > i$ where $x_j^* > x_j$. The index i must satisfy $i > s$ because for all x, $x_i \leqslant d_i$ and $x_i^* = d_i$ for $i = 1, \ldots, s$. Therefore, by definition of x^*, $x_j^* \leqslant x_i^* + 1$. Thus, we arrive at $x_j \leqslant x_i$ because $x_j < x_j^* \leqslant x_i^* + 1 < x_i + 1$. We can now define x' by modifying x, $x_i' = x_i - 1$ and $x_j' = x_j + 1$. In both cases, we increased the smaller element of x, which implies that x' lies in the

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

convex hull of x. Therefore by proposition due to Muirhead [17, p. 135], $x' \prec x$. After a finite number of steps of this transformation, we obtain $x = x^*$. Thus, we will obtain a sequence $x \succ x^1 \succ x^2 \cdots \succ x^*$. By Schur convexity of C_G, this sequence implies that $C_G(x) \geqslant C_G(x^*)$. □

5. Computation

We now consider the hardness of solving NANIP and introduce three algorithms for the problem, an Integer Programming Formulation, a Dynamic Programming algorithm and fast heuristics.

THEOREM 12 The NANIP is NP-hard.

Proof. NP-hardness can be established by reduction from the Max Independent Set problem. Given a graph G, we can find a maximal independent set by seeking the optimal solution to the instance of our problem with the same G and the function f defined by $f(0) = 0$ and $f(k) = 1$ for all $k \neq 0$. Using that f, any optimal permutation found by solving our problem will necessarily have the maximum possible number of nodes with cost 0, and the rest with cost 1. The nodes with cost 0 then necessarily form a maximal independent set, since if any two were adjacent then the latter of the two to be installed would have incurred a cost of $f(1) = 1$. □

Note that while this establishes that general NANIP is NP-hard, the restricted case of NANIP with a decreasing convex cost function remains an open problem.

5.1 *Integer programming formulation*

NANIP with a decreasing convex cost f can be formulated as an integer programme in the following way. For all $i \in V$ and $t \in \{1, \ldots, n\}$, introduce $X_{it} \in \{0, 1\}$ and let $X_{it} = 1$ iff $\sigma(t) = i$ and 0 otherwise. That is, $X_{it} = 1$ indicates that node i is installed at time t. Observe that i is installed before j if and only if $\sum_{t=1}^{T} X_{it} - \sum_{t=1}^{T} X_{jt} = 1$ for some $T \in \{1, 2, \ldots, n-1\}$. For each edge (i,j) (considering direction) introduce a weight $E_{ij} \in [0, 1]$ so that $E_{ij} + E_{ji} = 1$ for all $i, j \in V$. We will ensure that at all optimal solutions, E_{ij} takes integer values. Through E_{ij} variables we express $r(j, G, \sigma)$, the number of neighbours of a node j installed before j, i.e. $r(j, G, \sigma) = \sum_{i \in N(j)} E_{ij}$, where $N(j)$ denotes the set of neighbours of j. Lastly, let $l_d(x)$ define the downward sloping line through the points $(d-1, f(d-1))$ and $(d, f(d))$ with $d \in \{1, 2, \ldots, \}$. Collectively, those lines define a polyhedron whose extreme points coincide with the values of $f(d)$. Thus $l_d(x) = f(d) + (f(d) - f(d-1))(x - d)$.

NANIP is the problem given by

$$\begin{aligned} \min \quad & \sum_{i=1}^{n} c_i, \\ \text{s.t.} \quad & c_j \geqslant f(d) + (f(d) - f(d-1)) \left(\sum_{i \in N(j)} E_{ij} - d \right), \quad j \in V, \ \ d = 1, 2, \ldots, n-1, \end{aligned} \tag{11}$$

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

$$E_{ij} \geqslant \sum_{t=1}^{T} (X_{jt} - X_{it}), \quad T = 1, 2, \ldots, n-1, \;\; i \in V, j \in N(i)$$

$$E_{ij} + E_{ji} = 1 \quad \text{and} \quad E_{ij} \in [0,1], \qquad i \in V, \;\; j \in N(i)$$

$$\sum_{t=1}^{n} X_{jt} = 1 \quad \text{and} \quad \sum_{i=1}^{n} X_{it} = 1 \quad \text{and} \quad X_{it} \in \{0,1\} \qquad j \in V, \;\; t = 1, 2, \ldots, n. \tag{12}$$

Observe that because of the convexity of f, the points in the intersection of the inequalities (11) are the points above the convex hull of $\{f(d)\}_{d=1}^{n}$. At an optimal σ, the value of c_i would minimize $f(x)$ subject to $x \leqslant \sum_{i \in N(j)} E_{ij}$, so that the value of $c_i = f(r(i, G, \sigma))$.

Using CPLEX version 12.2 (IBM Corp.) we are able to solve NANIP on tree graphs with 50 nodes within one hour, and with longer running time as the number of edges increases. Convergence is accelerated when the basic NANIP is generalized to allow for realistic variation in the cost of a node ($f(d)$ becomes $f_i(d)$ for each $i \in V$). The speedup is likely due to the removal of symmetry in the problem, which must be considerable. For instance, in any graph on two or more nodes and any σ, swapping the first and second nodes in σ does not change the total cost.

5.2 *Dynamic programming algorithm*

Consider a generalization of NANIP which we call the Subgraph-Aided Network Installation Problem (SANIP). SANIP allows the cost $r(v_t, G, \sigma)$ of a node at time t to be an arbitrary function of the subgraph induced by the first $t-1$ nodes of σ, rather than just the neighbours of the node. This cost may even depend upon the installation time because it is the number of nodes $|V(H)|$.

SANIP instances can be solved by Algorithm 1, which exploits the fact that the cost of any node u depends only upon H and not how H was installed. Instead of considering the space of permutations on n nodes, which has size $n!$, it needs to consider merely the set of subsets of n nodes, which grows as 2^n. Concretely, to find an optimal installation of a subgraph on nodes $T \subset V$, it considers every decomposition of the form $T = S \cup \{u\}$. The algorithm maintains a table *TOPT* indexed by subsets $S \subset V$, and stores a least-cost sequence for installing S along with its cost. The maximum size of the *TOPT* table is $\binom{n}{n/2}$. In the pseudocode, the symbol $\vee$ on sequences denotes concatenation.

The algorithm uses $\Theta(2^{n/2})$ space (the table size peaks at iteration $n/2$ and then declines) and $\Theta(n2^n)$ time. We found that on a desktop computer the computation is tractable up to $n \approx 25$. While the space requirement might be improved, the running time cannot be improved beyond $\Omega(n2^n)$. The lower bound occurs because it is possible to have an instance where the cost of installing some node u is arbitrarily small if u is sequenced immediately after some subgraph H of G. Observe also that the specification of SANIP would in general use $\Theta(n2^n)$ space. To solve instances of larger size this algorithm could be replaced by a dynamical programming algorithm with a finite horizon.

5.3 *A linear-time heuristic*

Because practical instances of NANIP might be quite large, it is important to obtain highly scalable solvers. For example, a vocabulary training system might contain $n = 10^3$ words [18] while an infrastructure system might range from $n = 10^4$ [19] to several orders beyond that. Here, we consider heuristic approaches that can solve such large problems but do not guarantee optimality. A simple heuristic

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

Algorithm 1 Dynamic programming algorithm for construction of G

$TOPT[\{u\}] \leftarrow (u)$ for all $u \in V$.
for all $t \in \{2 \dots n\}$ **do**
  $TOPT' \leftarrow \varnothing$
  for all $S \in TOPT$ **do**
    for all $u \in V \setminus S$ **do**
      $T \leftarrow S \cup \{u\}$
      if $T \notin TOPT'$ **or** $cost(TOPT[S]) + c(u, S) < cost(TOPT'[T])$ **then**
        $TOPT'[T] \leftarrow TOPT[S] \vee (u)$
      end if
    end for
  end for
  free $TOPT$
  $TOPT \leftarrow TOPT'$
end for
Output $TOPT[V]$

Algorithm 2 Greedy Algorithm for NANIP

$\sigma_{\text{aux}} \leftarrow \varnothing$
If n=0 **Output** σ_{aux}
$\sigma_{\text{aux}} \leftarrow$ Random $u \in V$
while $|\sigma_{\text{aux}}| < n$ **do**
  $u \in \text{argmax}_{u \in V \setminus \sigma_{\text{aux}}} \{r(u, G, \sigma_{\text{aux}}) : u \in V \setminus \sigma_{\text{aux}}\}$, resolving ties arbitrarily.
  $\sigma_{\text{aux}} \leftarrow \sigma_{\text{aux}} \vee u$.
end while
Output σ_{aux}

approach is exemplified by Algorithm 2. It preferentially installs nodes that have the lowest installation cost, that is, nodes that have the most installed neighbours.

The algorithm as written runs in $O(Kn^2)$ time even if the degree of the nodes is bounded by a constant K: the external loop iterates over the variable n and the argmax operation takes $O(n)$ operations to search for the maximum, and $O(K)$ operations to update the $r(u, G, \sigma_{\text{aux}})$ values of the neighbours. An approach sometimes called lazy greedy could improve the running time of the inner loop to $O(K \log n)$. The lazy greedy idea is based on the observation that installing a node u only changes the r values of the neighbours of u, and not the rest of the nodes in G. Thus, one could cache all the current r values in a priority queue ordered by the r value. Recall that a priority queue could be implemented in a binomial heap, in which all operations, including look up, insert and delete run in $O(\log n)$ time. It follows then that the inner loop uses $O(1)$ time to find the maximum, and $O(K \log n)$ time to update the $r(u, G, \sigma_{\text{aux}})$ values. The overall running time for the lazy greedy algorithm is then $O(Kn \log n)$.

The cost of the solution found by this heuristic can be bounded when the cost function f is decreasing convex, as follows. In the worst case, the first node has cost $f(0)$, and all other nodes are constructed at cost $f(1)$. Often the bound can be made tighter, as follows.

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

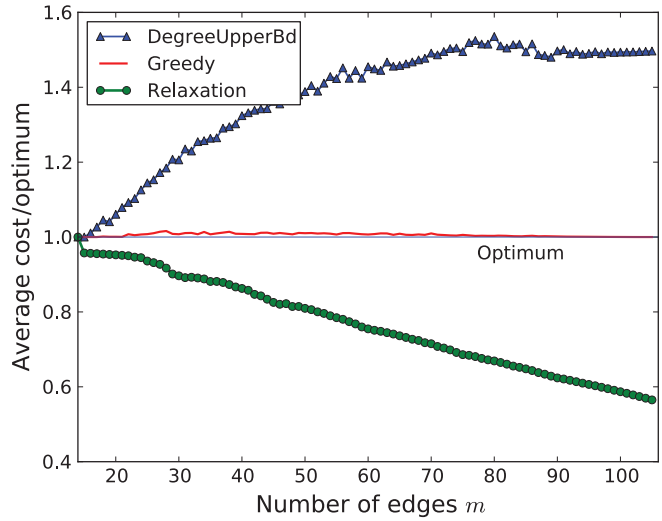


FIG. 3. Performance when solving convex NANIP problems with Algorithm 2 (Greedy) and tightness of optimality bounds. The greedy heuristic usually finds solutions indistinguishable from the optimum. The degree upper bound is from (13) and the relaxation bound is from (10).

PROPOSITION 13 Let f be decreasing convex and G a graph with node degrees $d_1 \leqslant d_2 \leqslant \cdots \leqslant d_n$ and $n \geqslant 3$. The cost of the solution σ found by Algorithm 2 is bounded by

$$C_G(\sigma) \leqslant f(0) + sf(1) + f(q) + f(d_{s+3}) + \cdots + f(d_n), \tag{13}$$

where $s \in \{0, 1, \ldots, n-1\}$ is the largest possible, such that $s + q + d_{s+3} + \cdots + d_n = m$ and $q \in \{1, 2, \ldots, d_{s+2}\}$.

In (13), the terms after $sf(1)$ occur only if $d_n > 1$.

The proof, using techniques similar to the proof of Theorem 11, is elementary. Given that we can bound from below the cost of the optimal solution (Corollary 7), one can further compute an optimality gap for any instance of NANIP.

The average performance of Algorithm 2 is illustrated by simulations in Fig. 3 and it is within 0.6% of the optimum. The result held true in our simulations across all edge densities with standard deviation (denoted by SD) of 0.4%. Therefore, and because it usually runs in near-linear time, Algorithm 2 might be appropriate to large-scale practical instances of NANIP. Also shown in Fig. 3 are the average value of the degree upper bound (from (13)) and the relaxation bound (from (10)) for the solution. The simulations construct a random connected graph on 15 nodes and m edges and the convex function $f(i) = 1/(1+i)$ for $i \in \mathbb{N}_0$. (The results are qualitatively similar for dozens of convex cost functions that we examined.) For each value of m, five random connected graphs are generated by taking a random tree on n nodes and adding $m - (n-1)$ edges to it at random. The greedy algorithm is scored based on the average over 10 runs on each graph. Small instance sizes ($n = 15$ nodes) were used because of the long-running time for finding the optimal value. It can be seen that the relaxed problem has a lower cost than NANIP, and the cost ratio is linear in the number of edges. NANIP thus incurs approximately a fixed cost per edge, which could be attributed to the requirement that one of the two incident nodes of every edge must be installed before the other.

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

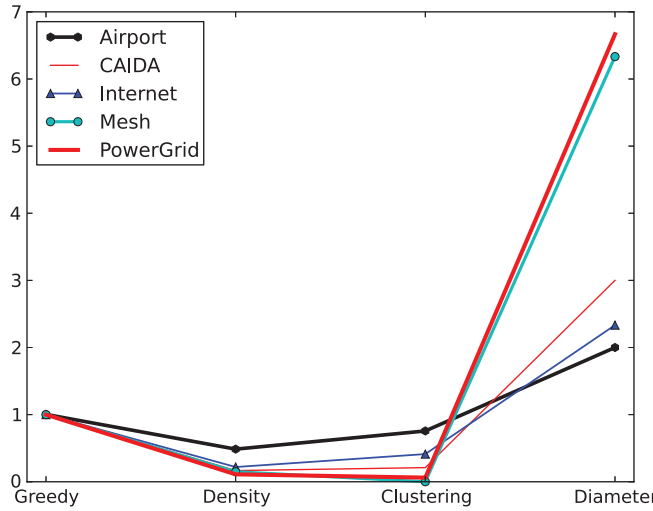


FIG. 4. Performance relative to optimum of Algorithm 2 (Greedy) on a set of infrastructure networks, and the properties of these networks. Like in the synthetic instances, the greedy heuristic achieves a high performance ratio (within 5×10^{-3} in all networks). See text for details of the simulations.

These results suggest that the greedy heuristic can solve convex NANIP with different edge densities to a high performance ratio. We have also analyzed the performance of the heuristic on realistic networks based on actual infrastructure data. It was found that the greedy method indeed has a high performance ratio (Fig. 4). We found no relationship between the performance ratio and characteristics of the graph including the density, clustering or diameter. To perform these simulations, we accessed several empirical topologies and from each topology we extracted three 20-node subnetworks using a breadth-first search from a randomly selected node. The greedy heuristic was run on each extract with $f(i) = 1/(1+i)$ and its solution value was divided by the optimal computed exactly with dynamic programming (this requires relatively small graphs). The plotted results show the average performance and network characteristics over the three extracts from a particular network. The clustering of the graph is defined here as $C = (1/|V|) \sum_{u \in V_G} (2\,T(u)/(\deg(u)(\deg(u)-1)))$, where $deg(u)$ is the degree of node u and $T(u)$ is the number of triangles through node u. 'Airport' is USA100 from [20], 'CAIDA' is the Autonomous Systems network (as-caida20071105) provided by the Stanford Network Analysis Project, 'Internet' is a large-scale representation of the Internet's Autonomous Systems imputed by the algorithm in [21], 'Mesh' is a 33×33 square mesh representative of many planned road networks and 'PowerGrid' is from [19]. As mentioned earlier, 20-node extracts were obtained from each network to make the exact solution tractable.

In addition to this greedy heuristic, we have examined a number of related algorithms. First, we have simulated a greedy heuristic which installs nodes based on their degree starting with the highest degree nodes. In our simulations, this degree-based heuristic did not outperform the cost-greedy heuristic (Algorithm 2). Second, we have considered the concave objective function $f(k) = (1 - k/n)^{1/2}$ (with the same setup of simulations as for the convex case). The heuristic based on degree achieved 1% (SD = 1%) of the optimal. Even better solutions were found by an 'anti-greedy' heuristic, which is like Algorithm 2, except that it has argmin instead of argmax. Thus, it installs the nodes that have the fewest existing installed neighbours, and thus the greatest installation cost. One expects the anti-greedy

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

heuristic to install a large independent set of nodes, and hence achieve savings through the concavity of the cost function f. Indeed, it achieved the best results for the case of concave cost function f, its solutions being just 0.1% larger than the optimum, on average (SD = 0.1%).

6. Conclusion

This paper introduces a new discrete optimization problem, the NANIP. We found that NANIP and its generalization could be solved using integer and dynamic programming, while a fast greedy approach exists for the case of a convex decreasing cost function. Two particularly promising directions for future work are: (i) to characterize the conditions under which in every optimal installation sequence the installed nodes at every time t would induce a connected subgraph, i.e. the installation expands from a single starting node and (ii) the parallel construction version of NANIP where multiple nodes can be installed per time step. We believe that both hold some practical and conceptual interest.

Acknowledgments

We thank Leonid Gurvits, Jason Johnson, Joel Lewis, Feng Pan, Aric Hagberg and Joel Nishimura for insightful conversations. We are indebted to an anonymous reviewer for suggesting a powerful majorization-based proof technique for Theorem 11 and also thank all anonymous reviewers whose reviews have greatly improved this study.

Funding

This work was supported by the Department of Energy at the Los Alamos National Laboratory under contract DE-AC52-06NA25396 through the Laboratory Directed Research and Development programme, and by the Defense Threat Reduction Agency ['Robust Network Interdiction Under Uncertainty' to A.G.] and by the National Institute of Standards and Technology (grant 60NANB10D128 to M.B.).

References

1. Guha, S., Moss, A., Naor, J. S. & Schieber, B. (1999) Efficient recovery from power outage (extended abstract). *Proceedings of the 31st Annual ACM Symposium on Theory of Computing, STOC'99*. New York, NY: ACM Press, pp. 574–582. ISBN 1-58113-067-8.
2. Nurre, S. G., Cavdaroglu, B., Mitchell, J. E., Sharkey, T. C. & Wallace, W. A. (2012) Restoring infrastructure systems: An integrated network design and scheduling (INDS) problem. *European J. Oper. Res.*, **223**, 794–806.
3. Lee, E. E., Mitchell, J. E. & Wallace, W. A. (2007) Restoration of services in interdependent infrastructure systems: a network flows approach. *IEEE Trans. Syst. Man Cybern. C: Appl. Rev.*, **37**, 1303–1317.
4. Adibi, M. M. & Fink, L. H. (1994) Power system restoration planning. *IEEE Trans. Syst.*, **91**, 22–28.
5. Bertoli, P., Cimatti, R., Slaney, J. & Thibaux, S. (2002) Solving power supply restoration problems with planning via symbolic model checking. *AIPS-02 Workshop on Planning via Model-Checking*, Toulouse, France, pp. 576–580.
6. Coffrin, C. Van Hentenryck, P. & Bent, R. (2011) Strategic stockpiling of power system supplies for disaster recovery. *Power and Energy Society General Meeting, 2011 IEEE*. New York: IEEE, pp. 1–8.
7. Van Hentenryck, P., Bent, R. & Coffrin, C. (2010) Strategic planning for disaster recovery with stochastic last mile distribution. *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems* (A. Lodi, M. Milano & P. Toth eds.). Lecturer Notes in Computer Science 6140. Berlin, Heidelberg: Springer, pp. 318–333. ISBN 978-3-642-13519-4.

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016

8. Novikoff, T. P., Kleinberg, J. M. & Strogatz, S. H. (2012) Education of a model student. *Proc. Natl Acad. Sci. USA*, **109**, 1868–1873.

9. Kleinberg, J. (2007) Cascading behavior in networks: algorithmic and economic issues. *Algorithmic Game Theory*, **24**, 613–632.

10. Held, M. & Karp, R. M. (1961) A dynamic programming approach to sequencing problems. *ACM'61: Proceedings of the 1961 16th ACM National Meeting*. New York, NY: ACM Press, pp. 71.201–71.204.

11. Mitchell, J. E. & Borchers, B. (1996) Solving real-world linear ordering problems using a primal-dual interior point cutting plane method. *Ann. Oper. Res.*, **62**, 253–276.

12. West, D. B. (2001) *Introduction to Graph Theory*. Upper Saddle River, NJ: Pearson Prentice Hall.

13. Schrijver, A. (2005) On the history of combinatorial optimization (till 1960). *Handb. Oper. Res. Manage. Sci.*, **12**, 1–68.

14. Erdős, P. & Rényi, A. (1960) On the evolution of random graphs. *Publ. Math. Instit. Hung. Acad. Sci.*, **5**, 17–67.

15. Molloy, M. & Reed, B. (1995) A critical point for random graphs with a given degree sequence. *Random Struct. Algorithms*, **6**, 161–179.

16. Bollobás, B. (2001) *Random Graphs*. Cambridge Studies in Advanced Mathematics, vol. 73. Cambridge: Cambridge University Press.

17. Marshall, A. W. & Olkin, I. (1979) *Inequalities: Theory of Majorization and Its Applications*. New York: Academic Press.

18. (2008) *Kaplan GRE Exam Verbal Workbook*. New York: Kaplan Publishing. ISBN 9781419552199.

19. Watts, D. J. & Strogatz, S. H. (1998) Collective dynamics of small-world networks. *Nature*, **393**, 440–442.

20. Colizza, V., Pastor-Satorras, R. & Vespignani, A. (2007) Reaction–diffusion processes and metapopulation models in heterogeneous networks. *Nat. Phys.*, **3**, 276–282.

21. Ron, D., Safro, I. & Brandt, A. (2011) Relaxation-based coarsening and multiscale graph organization. *Multiscale Model. Simul.*, **9**, 407–423.

Downloaded from http://comnet.oxfordjournals.org/ by guest on July 14, 2016